

WEEK 5 FRONTEND DEVELOPMENT

Full-Stack Application Development

Topics This Week

React Fundamentals | Components & Props | Tailwind CSS
State & Events | API Integration

Instructor: Dilovan Matini



Topics This Week

What we will learn in Week 5



- React Fundamentals
- Components & Props
- Tailwind CSS
- State & Events
- API Integration

DAY 1

React Fundamentals

JSX, Components, Project Setup, and Your First React App

Week 4 Recap

Backend Topics Covered

Backend Stack

- Node.js
- Express.js
- APIs & Routes
- JSON & Request/Response

Key Question

How do users interact with our APIs?

Through a frontend interface built with React — that's what Week 5 is all about.

What Is **React**?

React is a **JavaScript library** used to build user interfaces. Created by **Meta (Facebook)** in 2013. It allows developers to create interactive, component-based UIs efficiently.

Used By

- Facebook
- Netflix
- Instagram
- Airbnb



Why React?

Problems with Traditional HTML

- Large files
- Repeated code
- Difficult maintenance

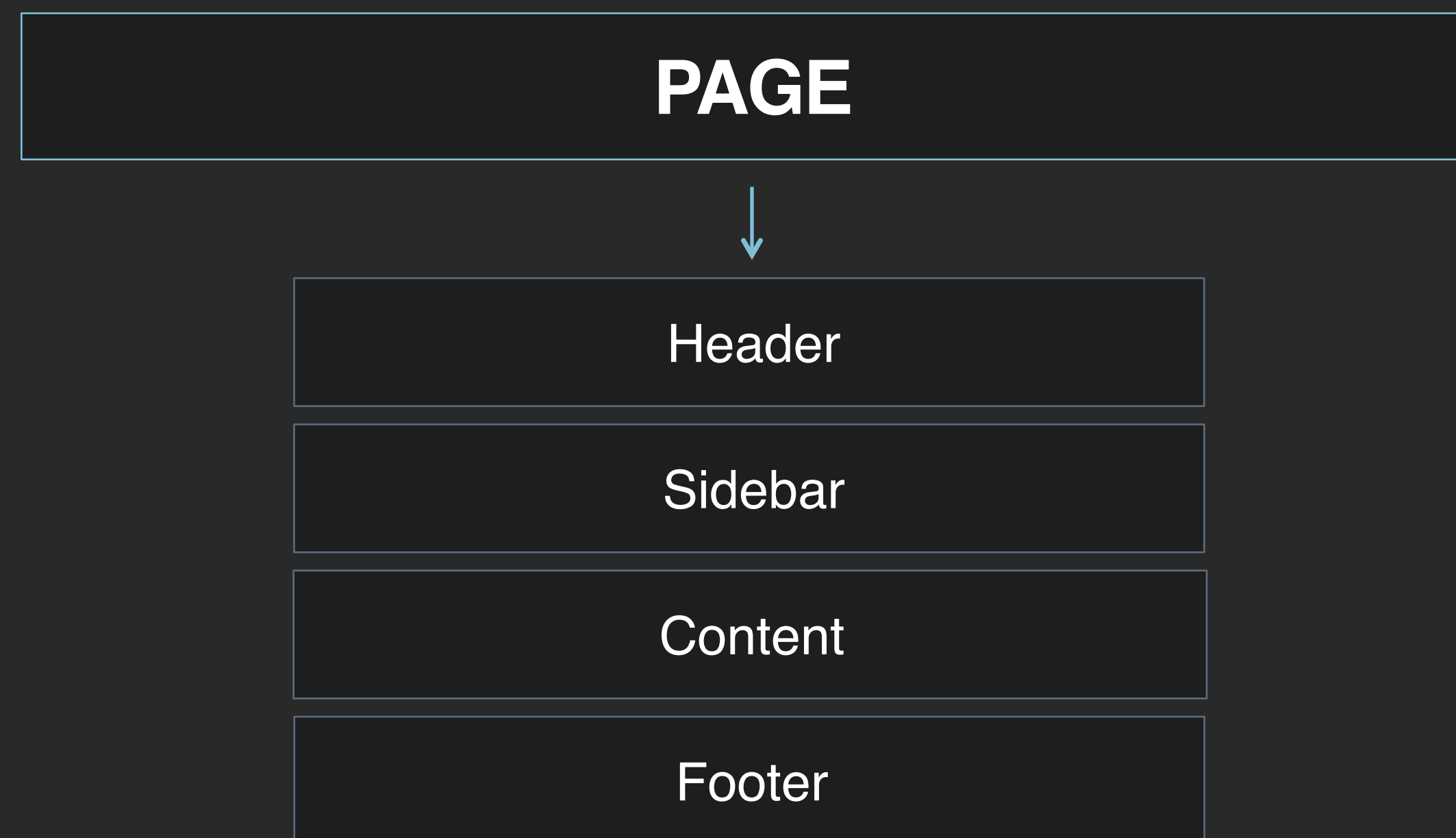
React Benefits

- Reusable components
- Faster development
- Better organization

Component-based architecture makes code **modular** and **maintainable**. Each piece of UI is self-contained and can be reused across the application.

Component-Based Development

Applications are built using components — reusable, self-contained pieces of UI



Each component manages its own logic and rendering. Components can be nested inside other components to build complex interfaces from simple building blocks.

Creating a React Project

Step-by-Step Setup

```
# Create project with Vite  
npm create vite@latest frontend  
  
# Choose: React + JavaScript  
  
# Install dependencies  
npm install  
  
# Run development server  
npm run dev
```



Vite

Vite provides a fast development server and modern build tooling. It replaces the older Create React App toolchain.

React Project Structure

frontend/

├── src/

| ├── **App.jsx**

| ├── main.jsx

| └── components/

├── public/

├── index.html

└── **package.json**

Important Files

- **App.jsx** — Root component that contains the main application layout and renders all other components.
- **main.jsx** — Entry point that renders the App component into the DOM using ReactDOM.
- **package.json** — Lists all project dependencies and scripts like npm run dev.

First React Component

A React component is a JavaScript function that returns JSX markup.

```
function App() {  
  return (  
    <h1>Hello React</h1>  
  );  
}  
  
export default App;
```

Code Breakdown

`function App()` — defines the component

`return` — outputs JSX to render

`export default` — makes it importable

JSX looks like HTML but works in JavaScript

JSX

What is JSX?

JSX looks like HTML but is actually JavaScript.

React converts JSX into JavaScript behind the scenes.

JSX allows embedding JavaScript expressions using **curly braces** {}.

```
// JSX looks like HTML
```

```
<h1>Hello Students</h1>
```

```
// But it's JavaScript
```

```
// Embedding JavaScript
```

```
const name = "Ali";
```

```
<h1>Hello, {name}!</h1>
```


Practice Exercise

Day 1

Your Task

- Create a **Header** component
- Create a **Footer** component
- Display them inside **App.jsx**

Learning Goals

This exercise reinforces component creation, export/import patterns, and JSX composition — the foundational skills of React development.

DAY 2

Components, Props & Tailwind

Building Reusable UI with Props and Utility-First CSS Styling



What Are Components?

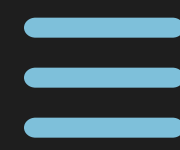
A component is a **reusable piece of user interface**.



Button



Card



Navbar



Footer

Components can be **nested**, **managed**, and **reused** across the application. Think of them as LEGO bricks — each piece is simple, but combined they build complex interfaces.

Creating Components

File Structure

components/Header.jsx

```
function Header() {  
  return <h1>CAPA Course</h1>;  
}  
  
export default Header;
```

Key Steps

- Create file in **components/** folder
- Write a function that returns JSX
- **Export** the component
- **Import** where needed

Using Components

Import Statement

```
import Header from './components/Header';
```

Use in JSX

```
function App() {  
  return (  
    <Header />  
  );  
}
```

Important Notes

Use **self-closing tags** for components without children: `<Header />`

Components can be used **multiple times** throughout the app.

Component names must start with a **capital letter**.

What Are Props?

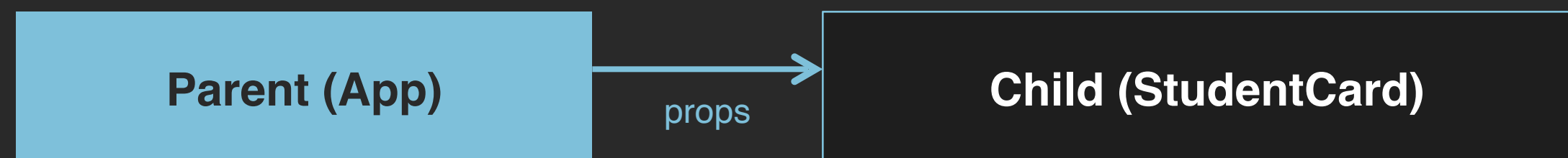
Props = Properties

Used to send data from parent to child components.

Example

```
<StudentCard name="Ali" />
```

Data Flow



Think of Props Like...

Props are like **function arguments** — they let you pass different data to the same component, making it dynamic and reusable.

Receiving Props

Method 1: props Object

```
function StudentCard(props) {  
  return (  
    <h2>{props.name}</h2>  
  );  
}
```

Method 2: Destructuring (Preferred)

```
function StudentCard({ name }) {  
  return (  
    <h2>{name}</h2>  
  );  
}
```

Best Practice

Destructuring is the preferred approach in modern React. It makes the code cleaner by explicitly showing which props the component expects.

Reusable Components

Same component. Different data.

Result

```
<StudentCard  
  name="Ali" />  
  
<StudentCard  
  name="Sara" />  
  
<StudentCard  
  name="Ahmed" />
```

Name: **Ali**

Name: **Sara**

Name: **Ahmed**

This demonstrates the power of **props** — **one component definition, infinite variations**. Write once, reuse everywhere.

Installing Tailwind CSS

Step 1: Install

```
npm install tailwindcss @tailwindcss/vite
```

Step 2: Update vite.config.js

```
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

Step 3: Update src/index.css

```
@import "tailwindcss";
```

Why Tailwind?

- **Fast styling** without writing custom CSS
- **Consistent design** system built-in
- **Responsive layouts** with built-in breakpoints

Tailwind Example

Code

```
<h1 className="text-5xl font-bold text-blue-600">Hello Students</h1>
```

Result

Hello Students

Class Breakdown

text-5xl → font size: 48px

font-bold → font weight: 700

text-blue-600 → color: blue

Each utility class applies a single CSS property.
Combine them to create any design.

Student Card Component

Multiple Props

```
<StudentCard  
  name="Ali"  
  age={22}  
  department="IT" />
```

Props Rules

- Multiple props can be passed to a single component
- **Numeric values** use curly braces: age={22}
- **Strings** can use quotes: name="Ali"

Component Preview

Ali

Age: 22

Department: IT

Practice Exercise

Day 2

Your Task

- Build a **Header** component with Tailwind
- Build a **Footer** component with Tailwind
- Build a **StudentCard** with props + Tailwind

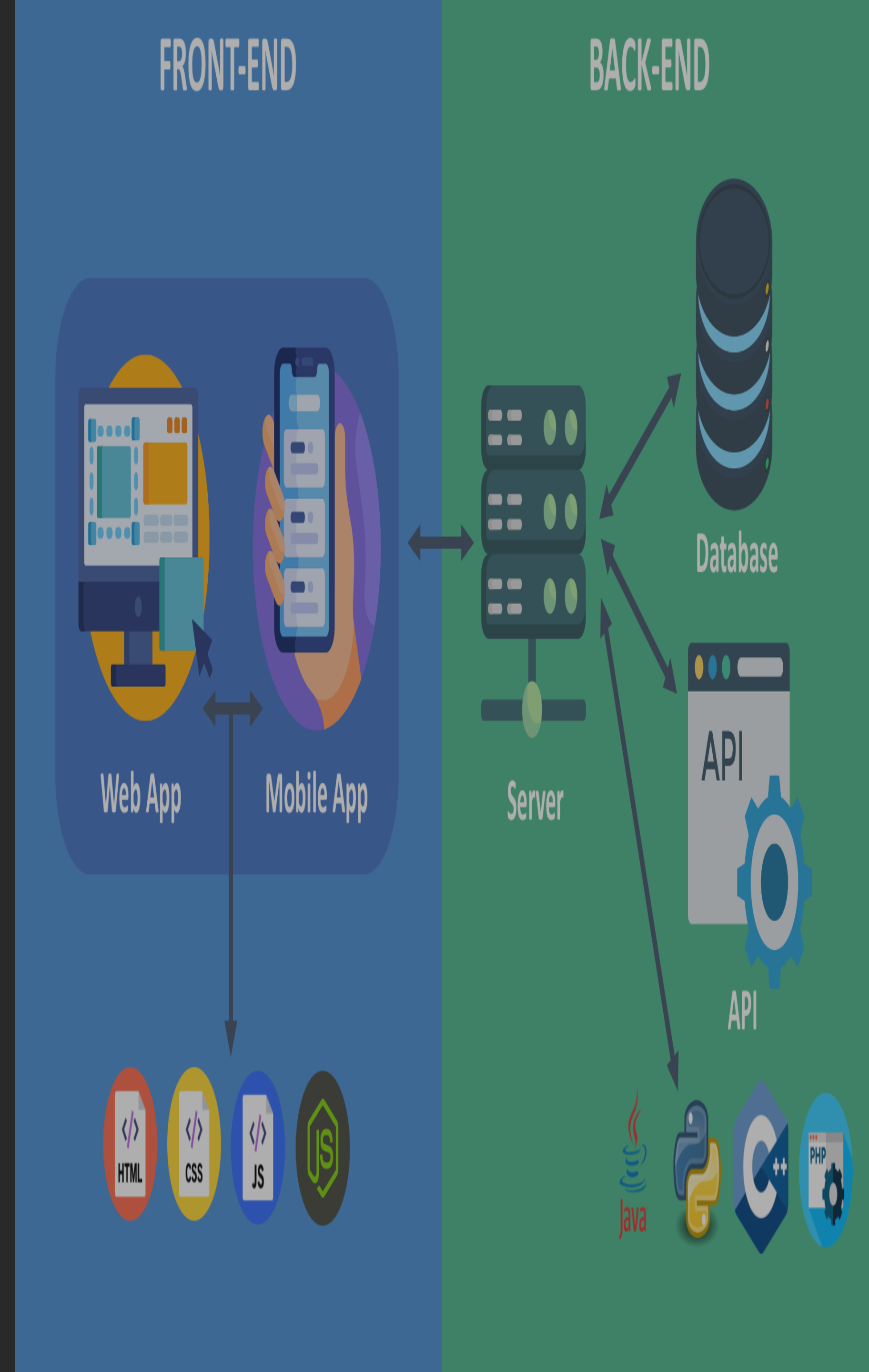
Learning Goals

This exercise combines component creation, props usage, and utility-first CSS styling — the core skills for building modern React interfaces.

DAY 3

State, Events & Backend

Interactive UI with State Management and API
Communication



What Is State?

State is **data that can change** while the application is running. It enables dynamic, interactive user interfaces.

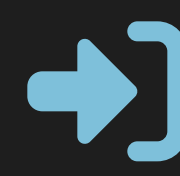
Common State Examples



Counter value



User name



Login status



Shopping cart

When state changes, React **automatically re-renders** the component to reflect the updated data. This is called **reactive rendering**.

useState Hook

Step 1: Import

```
import { useState } from 'react';
```

Step 2: Create State

```
const [count, setCount] = useState(0);
```

How It Works

`useState(0)` — initializes state with value 0

`count` — current state value

`setCount` — function to update the value

The array **destructuring** syntax `[ ]` extracts both values from `useState`.

Display State

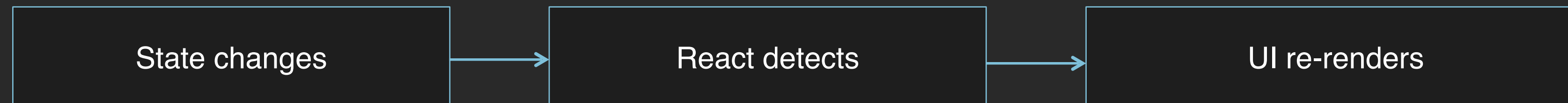
Render State in JSX

```
<h1>{count}</h1>
```

Live Preview

0

How React Updates the UI



Whenever state changes, React **automatically updates the UI**. This is the core of React's reactive programming model — you describe what the UI should look like for each state, and React handles the updates.

Update State

Button with onClick

```
<button
  onClick={() => setCount(count + 1)}
>
  Increase
</button>
```

Result

Every click increases the counter. State updates trigger re-renders automatically.

Important Rules

Never mutate state directly — always use the setter function.

State updates are **asynchronous** — React batches them for performance.

Each component has its own **independent state**.

Events

React events use **camelCase** syntax and are attached directly to JSX elements.

Common React Events

onClick

Fires when the user clicks an element. Used for buttons, links, and interactive elements.

onChange

Fires when the value of an input changes. Used for text fields, checkboxes, and selects.

onSubmit

Fires when a form is submitted. Used for login forms, search forms, and data entry.

Example

```
<button onClick={handleClick}>
```


Controlled Input

The input value is **controlled by React state** — the single source of truth.

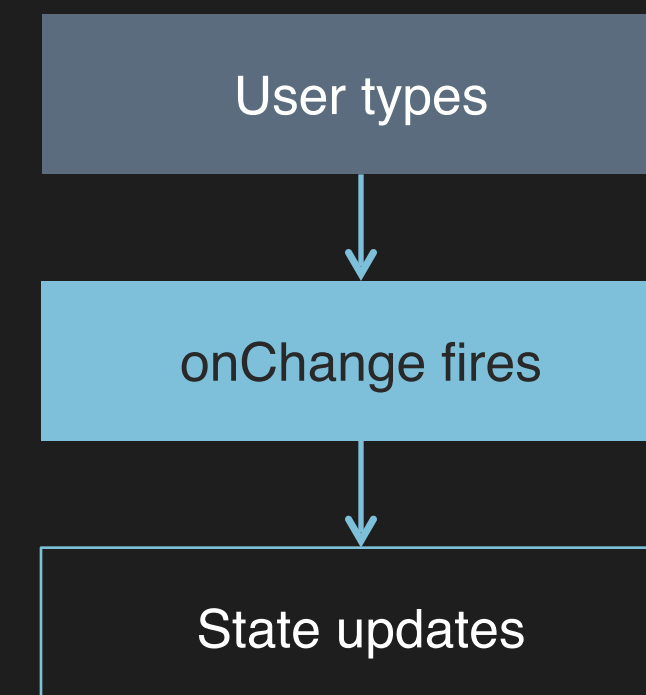
Step 1: Create State

```
const [name, setName] = useState("");
```

Step 2: Bind Input to State

```
<input  
  value={name}  
  onChange={(e) => setName(e.target.value)} />
```

Data Flow



Fetching Data From Backend

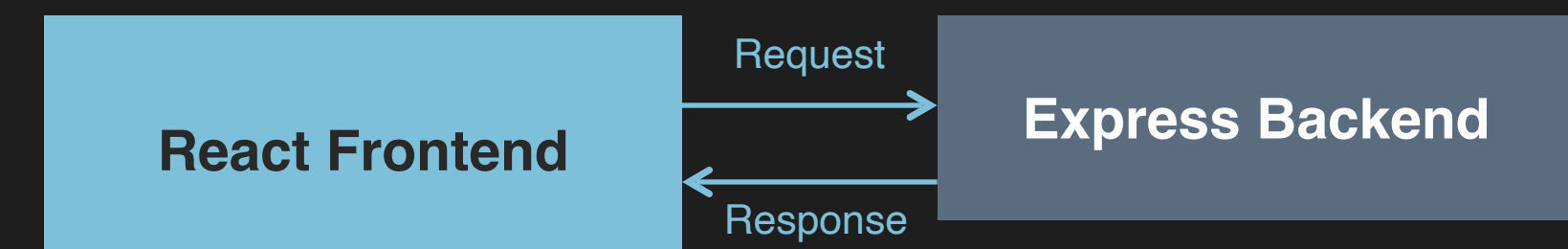
Backend API Endpoint

```
http://localhost:3000/users
```

Frontend fetch()

```
fetch('http://localhost:3000/users')
```

Full-Stack Flow



This bridges the frontend and backend together — the final piece of the full-stack puzzle.

Display API Data

Complete fetch Example

```
fetch('/users')  
  .then(response => response.json())  
  .then(data => {  
    console.log(data);  
  });
```

Process Flow

1 Send Request to backend API



2 Receive JSON response



3 Display Data in components

Understanding this **request-response flow** is essential for full-stack development. The frontend sends a request, the backend processes it, and returns data that the frontend displays.

Mini Project

Build a Student Management Frontend

 Header Component

 Footer Component

 Student Cards

 Fetch Students API

 Display Results

This project **integrates all concepts** learned this week: components, props, Tailwind CSS, state, events, and API fetching. It connects your React frontend to the backend API from Week 4.

Week 5 Wrap-Up

Topics Learned

- React
- JSX
- Components
- Props
- Tailwind CSS
- State
- Events
- API Integration

Project Phase 2

Build a React frontend that communicates with the backend API created during Week 4.

Next Week

Authentication, Routing, and Multi-Page Applications



Thank You

Week 5 — Frontend Development
Fundamentals

Instructor: **Dilovan Matini**

Course Provided By: **CAPA & GIZ**